Forcing-KV: Hybrid KV Cache Compression for Efficient Autoregressive Video Diffusion Models

Yicheng Ji[1,2] Zhizhou Zhong[2,3] Jun Zhang[1] Qin Yang[2] Xitai Jin[2]
Ying Qin[4] Wenhan Luo[3] Shuiyang Mao[2] Wei Liu[2] Huan Li[1,†]

[1]ZJU [2]Video Rebirth [3]HKUST [4]BJTU

† Corresponding Author

Abstract

Autoregressive (AR) video diffusion models adopt a streaming generation framework, enabling long-horizon video generation with real-time responsiveness, as exemplified by the *Self Forcing* training paradigm. However, existing AR video diffusion models still suffer from significant attention complexity and severe memory overhead due to the redundant key-value (KV) caches across historical frames, which limits scalability. In this paper, we tackle this challenge by introducing KV cache compression into autoregressive video diffusion. We observe that attention heads in mainstream AR diffusion models exhibit markedly distinct attention patterns and functional roles that remain stable across samples and denoising steps. Building on our empirical study of head-wise functional specialization, we divide the attention heads into two categories: *static heads*, which focus on transitions across autoregressive chunks and intra-frame fidelity, and *dynamic heads*, which govern inter-frame motion and consistency. We then propose FORCING-KV, a hybrid KV cache compression strategy that performs structured static pruning for static heads and dynamic pruning based on segment-wise similarity for dynamic heads. While maintaining output quality, our method achieves a generation speed of over **29** frames per second on a single NVIDIA H200 GPU along with **30%** cache memory reduction, delivering up to **1.35**× and **1.50**× speedups on LongLive and Self Forcing at 480P resolution, and further scaling to **2.82**× speedup at 1080P resolution.

Project Page: https://zju-jiyicheng.github.io/Forcing-KV-Page
Email Address: jiyicheng.cs@zju.edu.cn

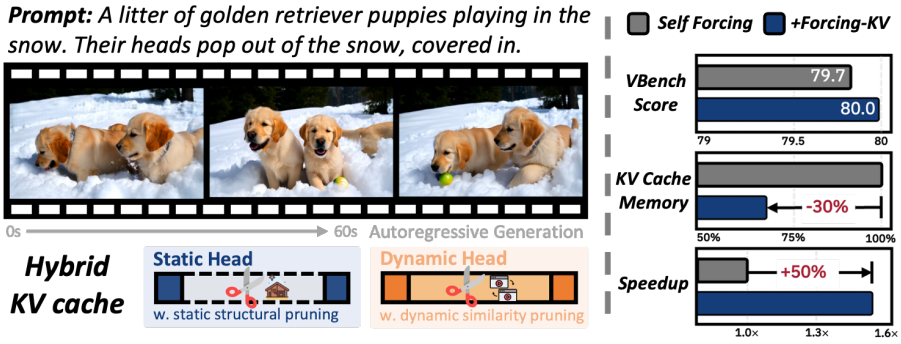

Figure 1: Overview of FORCING-KV. We apply static structural pruning and dynamic similarity pruning to different heads, accelerating inference, reducing cache memory while improving quality.

1 Introduction

Autoregressive (AR) video diffusion [1–6] has recently emerged as a compelling paradigm for efficient, streaming text-to-video generation. Unlike conventional bidirectional video diffusion

models [7–10] that denoise all frames simultaneously, AR video diffusion models produce video chunk by chunk, with each new chunk conditioned on previously generated video content via a key-value (KV) cache. This paradigm enables long-horizon, variable-length video generation with interactive inputs, while reducing both attention complexity and the latency to the first generated content. Mainstream approaches build upon the *Self Forcing* [1, 11] training paradigm, performing self-rollout during training to mitigate error accumulation, as exemplified by the broader family of "forcing" methods [1, 2, 11–19] that have shown strong performance.

However, existing mainstream AR video diffusion models still suffer from substantial attention complexity and severe memory overhead due to the heavy KV cache of historical chunks [2, 12, 20]. As video generation accumulates over time, the currently generated chunk is forced to attend to increasingly long and redundant visual context, which substantially reduces efficiency especially for long-horizon and high-resolution videos. For instance, generating a 30-second video at 1080P resolution with Self Forcing [1] takes over 2 minutes on a single NVIDIA H200 GPU, corresponding to a generation speed of 1.71 FPS considering only the overhead within the diffusion transformer (DiT). Moreover, the KV cache alone consumes more than 60 GB of GPU memory in this setting, which poses a major obstacle to deployment in memory-constrained scenarios.

To achieve real-time inference, studies have explored sparse attention [21, 22] and feature caching [23, 24] techniques for AR video diffusion models. Although effective, such methods neither reduce memory overhead nor operate on the KV cache, which is a distinctive structural component of AR video diffusion models. Recently, Dummy Forcing [25] observes that certain heads in AR diffusion models concentrate primarily on the currently generated chunk, and accordingly discards the historical context for those heads. However, it lacks a detailed analysis of the functional heterogeneity across attention heads, and its aggressive compression results in degraded temporal dynamics and discontinuity across chunks (i.e., flickering and broken transitions at chunk boundaries), as shown in Section 5. We posit that for AR video diffusion models, effective context utilization is key to both quality and efficiency. This raises a pivotal question:

Does autoregressive video diffusion model exhibit distinctive patterns in its KV cache utilization?

Our findings suggest an affirmative answer. We observe markedly distinct attention patterns and functional roles across the attention heads of AR video diffusion models. Through a series of careful empirical ablation studies in Section 3, we categorize the attention heads into the two categories. *Static heads* consistently attend to the current chunk and the most recent frame, which we denote as the *transition anchor frame*, to preserve intra-frame fidelity and visual continuity across autoregressive chunks. *Dynamic heads* capture inter-frame correspondences across the same spatial regions, governing subject consistency and motion dynamics. Moreover, we find that this head division remains stable across different samples and denoising steps, and generalizes broadly across multiple AR video diffusion models. Based on these observations, we propose FORCING-KV, a hybrid KV cache compression method for AR video diffusion models that decouples static structural patterns from dynamic context utilization.

FORCING-KV first introduces a one-shot, model-level **offline head profiling** procedure (see Section 4.1) that identifies static and dynamic heads based on frame-wise attention mass. Subsequently, we apply a hybrid KV cache compression strategy. For static heads, we adopt **static structural pruning** (see Section 4.2) to consistently preserve the transition anchor frame and prune distant frames. For dynamic heads, we employ **dynamic similarity pruning** (see Section 4.3), which computes segment-wise similarity between adjacent frames in the KV cache to retain temporally evolving content while pruning redundant and unchanged content. To summarize, our main contributions are:

(1) *Novel Pattern Discovery*: We uncover a universal head specialization pattern shared by mainstream autoregressive video diffusion models: transitions across autoregressive chunks are mediated by static heads that concentrate on the transition anchor frame, whereas long-horizon consistency and dynamics are sustained by dynamic heads through inter-frame attention.

(2) *Hybrid KV cache Compression*: Building upon this, we propose FORCING-KV, a compression strategy that preserves structurally critical content for static heads while applying dynamic similarity pruning for dynamic heads, decoupling static patterns from dynamic context utilization.

(3) *Extensive Experiments*: Evaluations across models, benchmarks, generation lengths, and resolutions show that FORCING-KV is both high-fidelity and efficient: While maintaining quality, FORCING-KV achieves up to **1.35**× and **1.50**× speedups along with **30%** cache memory reduction on LongLive and Self Forcing at 480P resolution, further scaling to **2.82**× at 1080P.

2 Related Work

Video Diffusion Models. Video diffusion models have evolved from bidirectional, one-shot generation to autoregressive, streaming generation. Early bidirectional video diffusion models [8–10] are typically built upon the Diffusion Transformer (DiT) [7] architecture, enabling high-quality and controllable video generation. To address the high cost of bidirectional denoising and support long-horizon video generation, a growing number of works turn to autoregressive diffusion modeling [6, 5, 4]. To further reduce denoising steps, CausVid [26] reformulates bidirectional diffusion into causal generation through distribution matching distillation. Self Forcing [1] mitigates train-test discrepancy by performing self-rollout during the training stage, and LongLive [2] further extends this framework through KV recaching and long-horizon fine-tuning. Krea-Realtime-14B [27] scales video generation to 14B parameters. More recently, a growing body of work [12, 13, 11, 15–17, 6, 18, 19, 28–31, 29] has focused on generating minute-long videos. Representative methods include Rolling Forcing [13], Reward Forcing [12], Infinite-Rope [15], and Self Forcing++ [11], most of which build upon the *Self Forcing* training paradigm. These efforts reflect a broader trend toward long-horizon video generation and the potential for a *train-long–test-long* strategy, in which KV cache size and memory overhead are critical factors for scalability and efficiency.

Efficient Video Generation. Video diffusion models are computationally expensive due to heavy attention computation and multi-step denoising. For bidirectional models, inference is typically accelerated through sparse attention [32–34], linear attention [35], quantization [36], and feature caching [37] techniques. Recently, several studies [38, 22, 21, 24, 23, 34, 39] have attempted to tailor these acceleration techniques to the characteristics of AR video diffusion models. However, AR video diffusion models natively rely on *KV cache* for streaming autoregressive inference, and most of the above methods do not alleviate cache size or memory overhead. Although KV cache compression has been widely studied in LLMs [40–43] and has been explored in autoregressive image generation [44, 45], it remains largely unexplored in AR video diffusion models. To compress the KV cache of AR video diffusion models, Dummy Forcing [25] observes that a subset of attention heads concentrates primarily on the currently generated chunk and exploits this property for compression. However, it lacks a detailed characterization of the attention patterns and functional roles of individual heads, and the aggressive compression leads to discontinuities across chunks and a drop in temporal dynamics. In contrast, we empirically identify the functional roles of different heads and perform hybrid compression based on their static and dynamic patterns, better preserving output quality.

3 Observation

In this section, we investigate the underlying principles of KV cache utilization in AR video diffusion models to motivate the compression strategy. We begin with intuitive observations of attention head patterns in Section 3.1, followed by empirical evidence that verifies the functional roles of different heads in Section 3.2, and finally investigate their stability and generalizability in Section 3.3.

3.1 Attention Head Pattern of Autoregressive Video Diffusion Models

Video diffusion models typically exhibit a spatial-temporal functional specialization [32, 33]. In AR video diffusion models, the introduction of KV cache allows this property to manifest over the evolving context of autoregressive generation.. This naturally raises the following question:

Question 1: *How do AR video diffusion models organize attention over spatiotemporal content during chunk-wise generation?*

To address this, we employ models including Wan2.1 [10], SkyReels-V2 [5], Self Forcing [1], and Longlive [2] to generate videos using VBench [46] prompts [1]. Through comparison across bidirectional, autoregressive, many-step and few-step video diffusion models, we categorize the attention heads into **static head** and **dynamic head**, and summarize the patterns in Figure 2:

Observation 1 (Static and Dynamic Head Pattern): *Static heads consistently attend to the current chunk and the most recent frame, preserving intra-frame fidelity and visual continuity across local autoregressive chunks. Dynamic heads capture the inter-frame evolution of corresponding regions to exploit long-range temporal context.*

[1] We provide the detailed attention map visualization (Figure 9) in Appendix B.

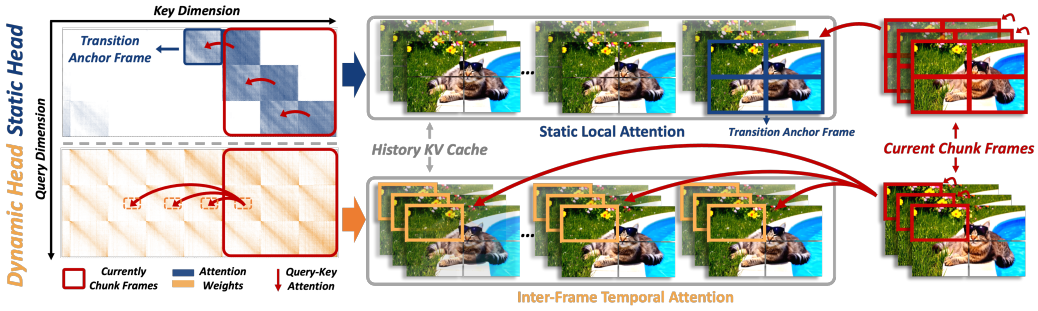

Figure 2: Attention head patterns in AR video diffusion models. Static heads focus on intra-frame dependencies and transitions across autoregressive chunks, whereas dynamic heads capture the inter-frame evolution of corresponding regions.

As illustrated in Figure 2, the static head primarily attends to local spatial frames. Consequently, its attention map exhibits a chunk-wise pattern, with consistent attention placed on the currently generated chunk. Concurrently, static heads also place particular attention on the most recent frame in the historical cache, which we refer to as the *transition anchor frame*. We regard this as a distinctive characteristic of autoregressive video diffusion models, where transitions across autoregressive chunks are primarily mediated through local, static attention to the transition anchor frame, rather than to the full set of historical frames. Through this attention pattern, the static head provides a structural scaffold for the video. We regard it as an invariant and static behavior in autoregressive video generation, independent of the prompts and the specific generated content.

In contrast, the dynamic head exhibits a diagonal stripe pattern with a constant interval in the KV cache. This phenomenon is highly interpretable: since both the number of frames per chunk and the number of tokens per frame are fixed, the same spatial region across different frames appears with a fixed stride along the key dimension. As a result, the dynamic head associates each generated region with information from the corresponding regions in historical frames (motion, object evolution), enabling the model to exploit long-range temporal context. Because different spatial regions evolve dynamically over the course of the video, we refer to this head pattern as dynamic.

3.2 Functional Properties of Static and Dynamic Heads

Having established an intuitive interpretation of the head patterns, we proceed to further examine the functional roles of the two types of heads.

Question 2: *What are the specific functional roles of the two types of heads, and what context in the KV cache is essential for them?*

We investigate this by conducting separate ablation studies that progressively mask the context accessible to each head until all historical frames are removed. Videos are generated using LongLive [2], a high-performing model for long-horizon video generation. For evaluation, we adopt 128 prompts from MovieGen [47] and use VBench-Long [48] as benchmark. Since existing metrics do not adequately capture flickering and broken transitions at chunk boundaries, we introduce an optical-flow-based metric, termed *chunk discontinuity* to measure [2]. It measures abrupt changes through the difference in optical flow between adjacent video frames. We summarize our empirical finding as:

Observation 2 (Functional Properties): *Static heads are crucial for visual continuity across autoregressive chunks while being insensitive to distant context. Dynamic heads govern subject consistency and motion dynamics, drawing on global context that is informative yet partially redundant.*

As shown in Figure 3 (a-c), as the number of visible historical frames is progressively reduced, both dynamic degree and consistency score gradually decline for dynamic heads, while remaining nearly unchanged for static heads. By contrast, masking the most recent frame (transition anchor frame) causes a sharp increase in chunk discontinuity for static heads, indicating significantly more abrupt transitions at chunk boundaries. We hypothesize that this effect further leads to degradation in other

[2]The detailed formulation and metric effectiveness are provided in Appendix A.

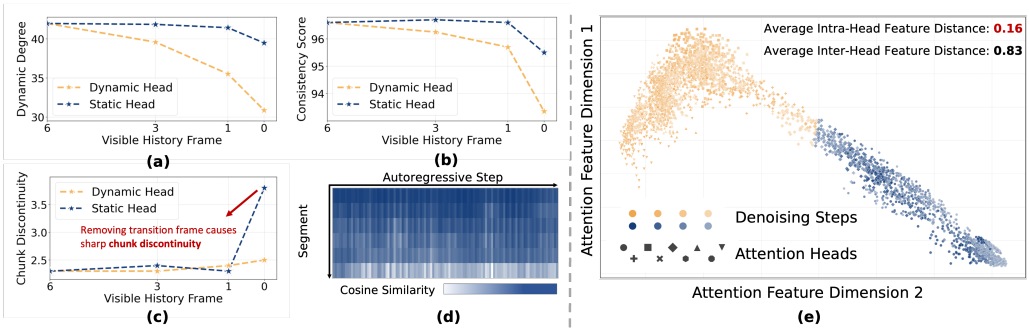

Figure 3: **Left:** (a-c) Gradually masking contextual information for dynamic heads leads to a progressive decline in dynamic degree and consistency, while masking the transition frame for static heads causes a sharp rise in chunk discontinuity, revealing different functional emphases. (d) The cosine similarity of key states of adjacent frames across different autoregressive steps and different frame segments. **Right:** (e) Principal component analysis (PCA) of attention features from a subset of attention heads, evaluated across one hundred prompt samples and four denoising steps. The observed head functioning is highly stable.

metrics. The above experiments also verify that transitions across autoregressive chunks are primarily mediated through attention to the transition anchor frame, rather than the full historical context.

Moreover, we observe that adjacent frames in autoregressive generation exhibit substantial regional similarity (potentially redundant), with generally high KV cache similarity that varies across different frame segments, as shown in Figure 3 (d). These insights provide empirical support for our hybrid compression scheme, that static heads are pruned statically while dynamic heads are pruned based on similarity, decoupling static local patterns from dynamic context utilization.

3.3 Stability of Head Properties

Furthermore, we conduct a statistical analysis of the above head properties to address the question:

Question 3: *Do the head properties remain stable, or do they exhibit substantial variation?*

To provide a comprehensive study, we experiment on LongLive with 100 standard VBench [46] prompts across all four denoising steps. For a random subset of heads, we extract the key states of each latent frame in the KV cache and compute frame-wise attention features. Based on the features, we visualize the distribution using principal component analysis (PCA) as shown in Figure 3 (e).

Observation 3 (Stability of Head Properties): *Head functional specialization remains stable across samples and denoising steps in its attention patterns.*

As shown in Figure 3(e), the features of each head form tightly clustered distributions across different samples and denoising steps, with average intra-head divergence (0.16) substantially smaller than average inter-head divergence(0.83). This provides a basis for effective head classification.

Discussion (Autoregressive Distinctiveness): Prior studies on bidirectional models also identify spatial-temporal head patterns [32]. Our observations differ in three important aspects. First, we uncover a unique dependency on transition anchor frames that is specific to autoregressive generation. Second, our observation is grounded in the KV cache, characterizing how the query chunk attends to previously generated chunks rather than fully bidirectional attention. Third, our compression scheme is based on temporal similarity in the KV cache rather than relying on a sparse attention pattern.

4 FORCING-KV

Motivated by the observations, we propose FORCING-KV, a hybrid compression scheme for autoregressive diffusion models, as depicted in Figure 4. We conduct a model-level **offline head profiling** in Section 4.1 that identifies static and dynamic heads. We then apply **static structural pruning** for static heads in Section 4.2 and **dynamic similarity pruning** for dynamic heads in Section 4.3.

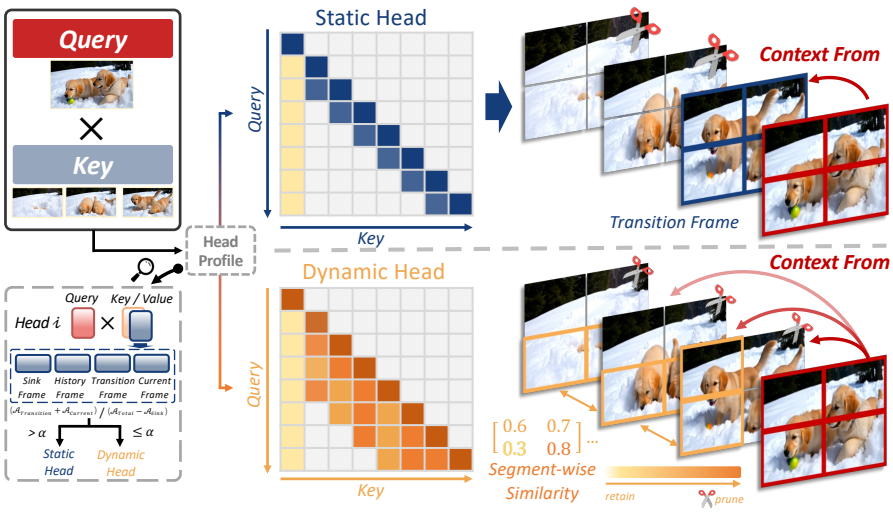

Figure 4: Overview of FORCING-KV. We perform offline head profiling to classify attention heads into Static and Dynamic. During inference, static heads are pruned leveraging the structural pattern, while dynamic heads are pruned adaptively based on segment-wise similarity of adjacent frames. For simplicity, we use one frame per chunk as an example.

4.1 Offline Head Profiling

Given the consistent functional behaviors of static and dynamic heads in *Observation 3*, we propose an offline head profiling strategy to categorize them before actual inference. According to the head pattern, the attention mass of static heads along the key dimension is concentrated on the currently generated chunk and the transition anchor frame, whereas the attention mass of dynamic heads is distributed more evenly across the entire attention window. This provides an intuitive criterion for head classification: utilizing the proportion of total attention mass assigned to the local static frames. Since some models apply special treatment to sink frames in their training recipes [2, 12, 15], we exclude the sink frames from this computation. Finally, given the per-head attention mass assigned to the entire attention window $\mathcal{A}_{Total}$, the generated chunk $\mathcal{A}_{Generate}$, the transition frame $\mathcal{A}_{Transition}$, and the sink frame $\mathcal{A}_{Sink}$, the head profiling metric is defined as:

$$\text{HeadType} = \begin{cases} \text{Static}, & \text{if } \dfrac{\mathcal{A}_{\text{Generate}} + \mathcal{A}_{\text{Transition}}}{\mathcal{A}_{\text{Total}} - \mathcal{A}_{\text{Sink}}} > \alpha, \\ \text{Dynamic}, & \text{otherwise.} \end{cases} \tag{1}$$

Here, α is a model-specific hyperparameter, and the classification can be completed within a single prompt. Notably, the metric aligns naturally with our subsequent compression strategy, where frames with lower accumulated attention mass are better eviction candidates, consistent with KV eviction methods such as H_2O [41]. In Section 5.3, we show that this simple criterion is sufficient to distinguish the majority of heads and is not sensitive to α, which promotes scalability.

4.2 Static Structural Pruning for Static Heads

In *Observation 1*, we show that static heads are highly sensitive to the transition anchor frame while underutilizing distant context. Therefore, we adopt a structured compression strategy for static heads by retaining the key and value states of the transition anchor frame and the current chunk to preserve intra-frame spatial structure and local chunk transitions. Given that each chunk contains C frames and each frame consists of F tokens, for the i-th AR step, the self-attention is formulated as:

$$\begin{aligned} \mathbf{O}_i^{\text{static}} = \text{Attention}\Big(&Q_{iCF:(i+1)CF}, \\ &\left[K_{sink},\ K_{(iC-1)F:iCF},\ K_{iCF:(i+1)CF}\right], \left[V_{sink},\ V_{(iC-1)F:iCF},\ V_{iCF:(i+1)CF}\right]\Big), \end{aligned} \tag{2}$$

where Q, K, and V denote the query, key, and value states, and K_{sink} denotes the key states of the sink frames. This formulation statically preserves the sink frames and the transition anchor frame for autoregressive chunks, and can be readily extended to frame-wise generation models as well.

4.3 Dynamic Similarity Pruning for Dynamic Heads

Dynamic heads assign high attention mass to regions separated by fixed intervals, corresponding to the same spatial locations across different frames. However, these segments differ substantially in their temporal evolution as shown in Figure 3 (d): some remain highly similar across frames with only limited variation (potentially background regions or static objects), whereas others undergo continuous changes due to motion, actions, or object evolution. Accordingly, we assess the redundancy of different segments during generation for dynamic compression.

Specifically, we first partition each latent frame into n segments and compute the segment-wise cosine similarity between corresponding segments in each frame and its next adjacent frame. Given a compression ratio r, we evict the top-k segments in each frame with the highest similarity values, preserving $(1-r)\%$ of all frame segments. Similar to Equation (2), let $K_{t,j}$ and $V_{t,j}$ denote the key and value state of the j-th segment in the t-th history frame, and we formulate the compression:

$$s_t^{(j)} = \mathrm{Cosine}\big(K_{t,j},\, K_{t+1,j}\big), \qquad \mathcal{I}_t^{\mathrm{keep}} = \mathrm{BottomK}\Big(\{s_t^{(j)}\}_{j=1}^{n},\, \lfloor rn \rfloor\Big). \tag{3}$$

Here, $s_t^{(j)}$ are cosine similarity values, and $\mathcal{I}_t^{\mathrm{keep}}$ are the indices of selected low-similarity segment to keep. Notably, for segment-wise similarity computation, we use only the key states over attention heads in the first block of the diffusion transformer [7] as a proxy, thereby avoiding the substantial computation of all blocks. We denote the compressed key and value states as $\widetilde{K}_{\mathcal{H}} = \bigcup_{t\in\mathcal{H}}\{K_{t,j} \mid j \in \mathcal{I}_t^{\mathrm{keep}}\}$ and $\widetilde{V}_{\mathcal{H}} = \bigcup_{t\in\mathcal{H}}\{V_{t,j} \mid j \in \mathcal{I}_t^{\mathrm{keep}}\}$, and the self-attention for dynamic head is denoted:

$$\begin{aligned}\mathbf{O}_i^{\mathrm{dynamic}} = \mathrm{Attention}\Big(&Q_{iCF:(i+1)CF},\\ &\Big[K_{\mathrm{sink}},\, \widetilde{K}_{\mathcal{H}},\, K_{iCF:(i+1)CF}\Big],\, \Big[V_{\mathrm{sink}},\, \widetilde{V}_{\mathcal{H}},\, V_{iCF:(i+1)CF}\Big]\Big).\end{aligned} \tag{4}$$

This design is motivated by the observation that adjacent frames in a video are often similar and therefore contain redundancy within a short temporal window [38]. As a result, removing highly similar segments introduces only minimal information loss. In Section 5.3, we empirically show that such a scheme offers an advantage in temporal dynamics over random and uniform token reduction.

5 Experiments

Models and Baselines. We conduct the experiments using mainstream AR video generation models including Self Forcing [1] and Longlive [2] [3]. We compare our method against both the full KV cache setting and representative KV cache compression baselines. StreamingLLM [40] serves as a naive baseline that uniformly retains sink and recent frames for all heads. Dummy Forcing [25] employs an aggressive local pruning strategy. We provide method implementation details in Appendix H.

Benchmarks and Evaluation Metrics. We evaluate both short and long video generation on VBench [46] and VBenchLong [48], and conduct a user study [4]. Specifically, we generate 5-second videos using 946 official VBench prompts and evaluate all 16 dimensions. For 30-second and 60-second videos, we adopt 128 prompts from MovieGen [47], weighting the total score using the standard VBench coefficients, consistent with previous work [3, 12]. All prompts are sampled with 5 different seeds. To quantify the continuity of chunk transition, we use *chunk discontinuity* metric as defined in Appendix A. For the efficiency metric, we calculate the frames generated per second (FPS) within the diffusion transformer (DiT) and the corresponding speedups on a single NVIDIA H200 GPU, together with the GPU memory usage of KV cache.

5.1 Main Results

Comparison with Full KV Cache. As shown in Tables 1 and 2 and Figure 5, we evaluate our method on both LongLive and Self Forcing across 5-second, 30-second and 60-second videos. Our method achieves 1.30× and 1.50× inference speedups on LongLive and Self Forcing for long video generation, and 1.35× and 1.44× speedups for short video generation. Through our head-wise hybrid

[3] Results on Krea-Realtime-14B [27] and interactive video generation on Longlive are in Appendices D and E.
[4] Setup, protocol, and screenshots of the user study are in Appendix C.

↑ indicates higher is better, ↓ indicates lower is better. ▲ highlights improved performance over Full KV.

Setting	Method	Efficiency Metrics		Specific Metrics		General Metrics					
		FPS↑	Speedup↑	Chunk Disc.↓	Dynamic Degree↑	**Total Score**↑	Imaging Quality↑	Subject Cons.↑	Background Cons.↑	Motion Smooth.↑	Aesthetic Quality↑
LongLive 60-second	Full KV	20.48	1.00×	2.6	42.40	80.23	68.84	97.82	96.82	98.77	61.47
	StreamingLLM	22.50	1.10×	2.5	40.89	80.28▲	69.40	97.91	96.88	98.75	61.78
	Dummy Forcing ($L = 1$)	28.06	1.37×	3.6	26.02	79.35	71.25	97.95	96.94	98.79	62.01
	Dummy Forcing ($L = 2$)	26.36	1.29×	2.9	34.10	79.44	70.71	97.38	96.52	98.45	61.40
	FORCING-KV (Ours)	26.71	1.30×	2.5▲	43.56▲	**80.43**▲	70.24	97.79	96.66	98.57	61.50
LongLive 30-second	Full KV	21.10	1.00×	2.4	42.54	80.38	68.91	97.99	96.93	98.80	61.67
	StreamingLLM	22.34	1.06×	2.4	40.93	80.38	69.35	96.06	96.98	98.78	61.92
	Dummy Forcing ($L = 1$)	27.45	1.30×	3.0	26.56	79.37	70.73	98.07	96.99	98.82	62.06
	Dummy Forcing ($L = 2$)	26.21	1.24×	2.3	33.90	79.61	70.65	97.64	96.64	98.54	61.73
	FORCING-KV (Ours)	26.77	1.27×	2.4	43.65▲	**80.65**▲	70.29	98.00	96.81	98.66	61.98
Self Forcing 30-second	Full KV	17.76	1.00×	3.4	46.86	79.72	67.63	97.20	96.38	98.14	61.14
	StreamingLLM	21.86	1.23×	2.8▲	54.50▲	80.06▲	67.90	96.93	96.16	98.10	59.67
	Dummy Forcing ($L = 1$)	27.75	1.56×	3.5	46.55	79.95▲	69.05	97.20	96.38	98.14	61.13
	Dummy Forcing ($L = 6$)	22.11	1.24×	3.4	50.47▲	79.78▲	68.83	96.62	96.03	98.03	59.97
	FORCING-KV (Ours)	26.65	1.50×	2.7▲	52.23▲	**80.07**▲	68.67	97.00	96.08	98.08	60.16

Table 1: Quantitative results on efficiency and quality on **long video generation** with VBenchLong.

Setting	Method	Efficiency Metrics		Specific Metrics		General Metrics		
		FPS↑	Speedup↑	Chunk Disc.↓	Dynamic Degree↑	**Total Score**↑	Quality Score↑	Semantic Score↑
LongLive 5-second	Full KV	21.85	1.00×	2.1	40.28	83.19	83.63	81.41
	StreamingLLM	24.56	1.12×	2.1	43.33▲	82.85	83.29	81.09
	Dummy Forcing ($L = 1$)	29.69	1.36×	2.4	37.22	82.83	83.33	80.86
	Dummy Forcing ($L = 2$)	28.36	1.30×	2.1	36.67	83.15	83.73	80.85
	FORCING-KV (Ours)	29.58	1.35×	2.1	45.56▲	**83.23**▲	83.84	80.80
Self Forcing 5-second	Full KV	19.56	1.00×	2.1	66.39	83.91	84.71	80.70
	StreamingLLM	23.36	1.19×	2.4	65.00	83.80	84.52	80.89
	Dummy Forcing ($L = 1$)	28.31	1.45×	2.6	64.44	83.87	84.64	80.81
	Dummy Forcing ($L = 6$)	25.41	1.30×	2.4	63.33	83.79	84.50	80.96
	FORCING-KV (Ours)	28.18	1.44×	2.1	69.17▲	**83.98**▲	84.82	80.61

Table 2: Quantitative results on **short video generation** with VBench.

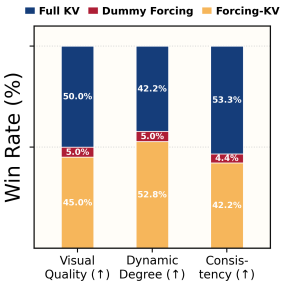

Figure 5: User study.

cache compression, only ∼27% and ∼46% of the KV cache participate in self-attention computation for Self Forcing and Longlive. At the same time, FORCING-KV maintains comparable or slightly improved performance on VBench (80.43 vs. 80.23). In the user study, FORCING-KV also achieves comparable visual quality (45.0% vs. 50.0%) and stronger temporal dynamics (52.8% vs. 42.2%). We assume that most existing base models treat all heads uniformly, forcing certain heads (i.e., static heads) to attend to distant context, but such capability may not be sufficiently learned. As a result, compressing the context for these heads can even lead to a positive effect. This also explains why StreamingLLM can maintain the performance even after evicting partial distant tokens. The results show the presence of redundancy in KV cache utilization, lending support to our observations.

Comparison with Other Compression Strategies. ① Vs. StreamingLLM: With its sliding window design closely aligned with the training regime of the base model, StreamingLLM serves as a competitive baseline. However, FORCING-KV achieves head-level decomposition, which leads to substantially higher compression ratios and resulting speedups (1.50× vs. 1.23×). ② Vs. Dummy Forcing: While achieving comparable speedups, FORCING-KV consistently achieves substantially much higher quality metrics. Specifically, our compression is grounded in the observations from Section 3 and preserves the transition anchor frame, whereas Dummy Forcing does not. This results in markedly lower chunk discontinuity (2.5 vs. 3.6 and 2.7 vs. 3.5). Moreover, the aggressive compression of Dummy Forcing leads to a pronounced degradation in dynamic degree (26.02 vs. 43.56), even under conservative settings (34.10) when $L = 6$. Though it achieves higher image quality in some cases, this may reflect the benchmark's preference for static content. Notably, chunk continuity and temporal dynamics have a substantial impact on perceptual quality, which also explains why our method shows a clear advantage over Dummy Forcing in the user study (45.0% vs. 5.0%) [5].

5.2 Scaling Law for Attention Window Size and Resolution

In Tables 1 and 2, the speedup gains achieved by FORCING-KV are bounded by the KV cache size. However, we empirically show that the acceleration benefits of FORCING-KV become increasingly

[5] Qualitative examples are provided in Appendix L.

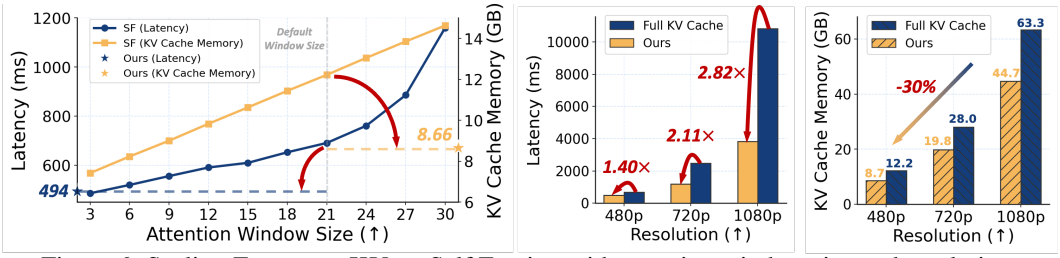

Figure 6: Scaling FORCING-KV on Self Forcing with attention window size and resolution.

pronounced as the attention window and resolution grow. In Figure 6, we show the latency and memory of Self Forcing. With the attention window and video resolution increasing, the KV cache grows accordingly, causing the attention computation to scale quadratically and the memory consumption to scale linearly. Therefore, for the same compression ratio, the resulting speedup becomes more significant. Under this trend, FORCING-KV delivers increasing gains, rising from 1.40× to 2.82× with a memory reduction of ∼30%. Notably, the effective KV size is even smaller since dynamic heads retain historical frames for similarity computation. A potential retrieval strategy may further reduce memory usage. We argue that high-resolution video generation and longer video contexts are promising future directions, which further highlight the potential of our method.

5.3 Ablation Study

In Table 3, we conduct separate ablation studies on the effectiveness of the head profiling strategy, the hybrid compression design, and the dynamic similarity pruning strategy. Unless otherwise specified, all experiments are performed on 30-second video generation with LongLive [6].

Effectiveness of Head Profiling. Owing to the clear distinction among head types and their stability, the simple profiling strategy of FORCING-KV is sufficient to identify the majority of heads and achieves performance close to that of a strong manual profiling baseline (80.65 vs. 80.71). We also include a random profiling baseline. Because the cache retained for dynamic heads also subsumes the static portion, the degradation under random profiling mainly arises when dynamic heads are misclassified as static, causing dynamic degree to drop (40.72 vs. 43.65). This indicates the necessity of meaningful head classification. In addition, we find that head profiling of FORCING-KV is insensitive to the hyperparameter α. Decreasing α from 0.8 to 0.5 classifies more heads as static heads, which leads to only a slight drop in dynamic degree.

Effectiveness of Hybrid Compression. FORCING-KV retains the transition anchor frame for static heads and preserves the cache segments for dynamic heads. We separately prune the corresponding cache of each head type to study the effect of hybrid head modeling, denoted as *w/o static-head cache* and *w/o dynamic-head cache*. As shown in Table 3, pruning the cache of static head leads to a substantial increase in chunk discontinuity (4.1 vs. 2.4), which in turn degrades the overall score. In contrast, pruning all cache of dynamic head mainly reduces the dynamic degree (40.78 vs. 43.65), consistent with our finding in Section 3.2. This indicates that the two components retained by our compression strategy are indeed effective, and that they correspond to their function roles.

Effectiveness of Dynamic Similarity Pruning. To validate the effectiveness of the dynamic similarity pruning strategy in FORCING-KV, we conduct comparisons against other pruning criteria including random token pruning and uniform token pruning, as shown in Figure 7. Our method consistently achieves higher dynamic degree, and improves temporal dynamics as the token budget increases. We attribute this advantage to two factors. First, adjacent frames in autoregressive generation often exhibit substantial similarity as visualized in Figure 3 (d), making them suited for cross-frame pruning. Second, our method operates in a segment-wise manner, which is better aligned with the continuity of video content than discrete token-wise strategies. This design is also consistent with our observation in Figure 2 that the stripe patterns of dynamic heads exhibit a certain width [7].

[6] Additional ablation results for 5-second video generation on LongLive and Self Forcing are in Appendix G.
[7] We integrate FORCING-KV with quantization in Appendix F.

Method	Chunk Disc.↓	Dynamic Degree↑	Total Score↑
Head Profiling			
FORCING-KV ($\alpha = 0.8$)	2.4	43.65	80.65
FORCING-KV ($\alpha = 0.5$)	2.4	42.87	80.63
Random Profiling	2.5	40.72	80.44
Human Profiling	2.3	44.44	80.71
KV Cache Compression			
FORCING-KV	2.4	43.65	80.65
w/o static-head cache	4.1	42.58	79.57
w/o dynamic-head cache	2.6	40.78	80.25

Table 3: Ablation study of head profiling strategy and hybrid KV cache compression on LongLive (30-second video generation).

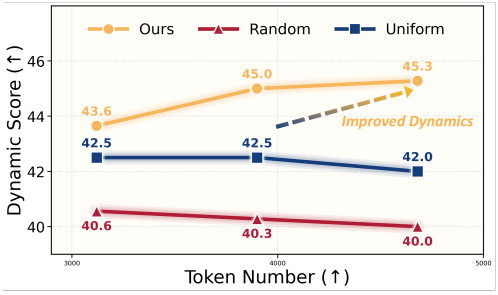

Figure 7: Dynamic score comparison of random token pruning, uniform token pruning, and our proposed similarity pruning.

6 Conclusion

We presented FORCING-KV, a hybrid KV cache compression framework for autoregressive video diffusion models. We begin by uncovering a universal head specialization pattern shared across mainstream autoregressive video diffusion models, which naturally motivates our compression strategy. While maintaining output quality, FORCING-KV achieves a generation speed of over 29 FPS, delivering up to 1.35× and 1.50× speedups together with 30% cache memory reduction, and scales effectively with attention window size and resolution, reaching up to 2.82× acceleration. Our work reveals the underlying mechanisms of KV cache utilization in autoregressive video generation, providing new empirical insights and compression techniques into efficient video cache utilization.

7 Limitation and Future Works

While our hybrid compression framework yields substantial gains in efficiency and performance for AR video diffusion models, it can be further improved in the following aspects. First, since mainstream open-source autoregressive video diffusion models are currently trained under the *Self Forcing* paradigm, our observations are primarily based on existing model families, including Self Forcing [1], Longlive [2], SkyReels-V2 [5], Krea-Realtime-14B [27] and the broader family of "forcing" models [12, 11, 15, 19, 13, 16, 49]. Nevertheless, we believe that these observations are closely tied to the fundamental principles of autoregressive generation, and that evaluating them on future autoregressive models would be an interesting direction for further study. Second, as a training-free method, investigating KV cache reduction during the training stage of autoregressive video models is left for future work, which could potentially support longer context window through fine-tuning.

References

[1] X. Huang, Z. Li, G. He, M. Zhou, and E. Shechtman, "Self Forcing: Bridging the Train-Test Gap in Autoregressive Video Diffusion," *NeurIPS*, 2025.

[2] S. Yang, W. Huang, R. Chu, Y. Xiao, Y. Zhao, X. Wang, M. Li, E. Xie, Y. Chen, Y. Lu *et al.*, "Longlive: Real-time Interactive Long Video Generation," *ICLR*, 2026.

[3] T. Yin, Q. Zhang, R. Zhang, W. T. Freeman, F. Durand, E. Shechtman, and X. Huang, "From Slow Bidirectional to Fast Autoregressive Video Diffusion Models," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 22 963–22 974.

[4] H. Teng, H. Jia, L. Sun, L. Li, M. Li, M. Tang, S. Han, T. Zhang, W. Zhang, W. Luo *et al.*, "Magi-1: Autoregressive video generation at scale," *arXiv preprint arXiv:2505.13211*, 2025.

[5] G. Chen, D. Lin, J. Yang, C. Lin, J. Zhu, M. Fan, H. Zhang, S. Chen, Z. Chen, C. Ma *et al.*, "Skyreels-v2: Infinite-length film generative model," *arXiv preprint arXiv:2504.13074*, 2025.

[6] L. Zhang, S. Cai, M. Li, G. Wetzstein, and M. Agrawala, "Frame Context Packing and Drift Prevention in Next-Frame-Prediction Video Diffusion Models," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[7] W. Peebles and S. Xie, "Scalable Diffusion Models with Transformers," in *2023 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, 2023, pp. 4172–4182.

[8] B. Lin, Y. Ge, X. Cheng, Z. Li, B. Zhu, S. Wang, X. He, Y. Ye, S. Yuan, L. Chen, T. Jia, J. Zhang, Z. Tang, Y. Pang, B. She, C. Yan, Z. Hu, X. Dong, L. Chen, Z. Pan, X. Zhou, S. Dong, Y. Tian, and L. Yuan, "Open-Sora Plan: Open-Source Large Video Generation Model," 2024.

[9] W. Kong, Q. Tian, Z. Zhang, R. Min, Z. Dai, J. Zhou, J. Xiong, X. Li, B. Wu, J. Zhang, K. Wu, Q. Lin, J. Yuan, Y. Long, A. Wang, A. Wang, C. Li, D. Huang, F. Yang, H. Tan, H. Wang, J. Song, J. Bai, J. Wu, J. Xue, J. Wang, K. Wang, M. Liu, P. Li, S. Li, W. Wang, W. Yu, X. Deng, Y. Li, Y. Chen, Y. Cui, Y. Peng, Z. Yu, Z. He, Z. Xu, Z. Zhou, Z. Xu, Y. Tao, Q. Lu, S. Liu, D. Zhou, H. Wang, Y. Yang, D. Wang, Y. Liu, J. Jiang, and C. Zhong, "HunyuanVideo: A Systematic Framework For Large Video Generative Models," 2025.

[10] T. Wan, A. Wang, B. Ai, B. Wen, C. Mao, C.-W. Xie, D. Chen, F. Yu, H. Zhao, J. Yang *et al.*, "Wan: Open and advanced large-scale video generative models," *arXiv preprint arXiv:2503.20314*, 2025.

[11] J. Cui, J. Wu, M. Li, T. Yang, X. Li, R. Wang, A. Bai, Y. Ban, and C.-J. Hsieh, "Self-Forcing++: Towards Minute-Scale High-Quality Video Generation," *ICLR*, 2026.

[12] Y. Lu, Y. Zeng, H. Li, H. Ouyang, Q. Wang, K. L. Cheng, J. Zhu, H. Cao, Z. Zhang, X. Zhu *et al.*, "Reward forcing: Efficient streaming video generation with rewarded distribution matching distillation," *CVPR*, 2026.

[13] K. Liu, W. Hu, J. Xu, Y. Shan, and S. Lu, "Rolling forcing: Autoregressive long video diffusion in real time," *ICLR*, 2026.

[14] J. Yi, W. Jang, P. H. Cho, J. Nam, H. Yoon, and S. Kim, "Deep forcing: Training-free long video generation with deep sink and participative compression," *arXiv preprint arXiv:2512.05081*, 2025.

[15] H. Yesiltepe, T. H. S. Meral, A. K. Akan, K. Oktay, and P. Yanardag, "Infinity-rope: Action-controllable infinite video generation emerges from autoregressive self-rollout," *CVPR*, 2025.

[16] S. Xiao, X. Zhang, D. Meng, Q. Wang, P. Zhang, and B. Zhang, "Knot Forcing: Taming Autoregressive Video Diffusion Models for Real-time Infinite Interactive Portrait Animation," *arXiv preprint arXiv:2512.21734*, 2025.

[17] Y. Huang, H. Guo, F. Wu, S. Zhang, S. Huang, Q. Gan, L. Liu, S. Zhao, E. Chen, J. Liu *et al.*, "Live avatar: Streaming real-time audio-driven avatar generation with infinite length," *arXiv preprint arXiv:2512.04677*, 2025.

[18] W. Li, W. Pan, P.-C. Luan, Y. Gao, and A. Alahi, "Stable Video Infinity: Infinite-Length Video Generation with Error Recycling," in *International Conference on Learning Representations*, 2026.

[19] J. Cui, J. Wu, M. Li, T. Yang, X. Li, R. Wang, A. Bai, Y. Ban, and C.-J. Hsieh, "LoL: Longer than Longer, Scaling Video Generation to Hour," *arXiv preprint arXiv:2601.16914*, 2026.

[20] H. Chen, C. Xu, X. Yang, X. Chen, and C. Deng, "Past-and Future-Informed KV Cache Policy with Salience Estimation in Autoregressive Video Diffusion," *arXiv preprint arXiv:2601.21896*, 2026.

[21] C. Lv, Y. Shi, Y. Huang, R. Gong, S. Ren, and W. Wang, "Light Forcing: Accelerating Autoregressive Video Diffusion via Sparse Attention," *arXiv preprint arXiv:2602.04789*, 2026.

[22] K. Agarwal, Z. Chen, C. Luo, Y. Chen, H. Zheng, X. Huang, A. Rudra, and B. Chen, "MonarchRT: Efficient Attention for Real-Time Video Generation," *arXiv preprint arXiv:2602.12271*, 2026.

[23] Y. Ma, X. Zheng, J. Xu, X. Xu, F. Ling, X. Zheng, H. Kuang, H. Li, X. WANG, X. Xiao *et al.*, "Flow Caching for Autoregressive Video Generation," in *The Fourteenth International Conference on Learning Representations*, 2026.

[24] D. Samuel, I. Tzachor, M. Levy, M. Green, G. Chechik, and R. Ben-Ari, "Fast Autoregressive Video Diffusion and World Models with Temporal Cache Compression and Sparse Attention," *arXiv preprint arXiv:2602.01801*, 2026.

[25] H. Guo, Z. Jia, J. Li, B. Li, Y. Cai, J. Wang, Y. Li, and Y. Lu, "Efficient Autoregressive Video Diffusion with Dummy Head," *arXiv preprint arXiv:2601.20499*, 2026.

[26] T. Yin, Q. Zhang, R. Zhang, W. T. Freeman, F. Durand, E. Shechtman, and X. Huang, "From Slow Bidirectional to Fast Autoregressive Video Diffusion Models," in *CVPR*, 2025.

[27] E. Millon, "Krea Realtime 14B: Real-time Video Generation," 2025.

[28] X. Xiang, Z. Duan, G. Zhang, H. Zhang, Z. Gao, J. Wu, S. Zhang, T. Wang, Q. Fan, and C. Guo, "Pathwise Test-Time Correction for Autoregressive Long Video Generation," *arXiv preprint arXiv:2602.05871*, 2026.

[29] S. Cai, W. Nie, C. Liu, J. Berner, L. Zhang, N. Ma, H. Chen, M. Agrawala, L. Guibas, G. Wetzstein *et al.*, "Mode Seeking meets Mean Seeking for Fast Long Video Generation," *arXiv preprint arXiv:2602.24289*, 2026.

[30] M. L. Team, X. Cai, Q. Huang, Z. Kang, H. Li, S. Liang, L. Ma, S. Ren, X. Wei, R. Xie, and T. Zhang, "LongCat-Video Technical Report," *arXiv preprint arXiv:2510.22200*, 2025.

[31] S. Yuan, Y. Yin, Z. Li, X. Huang, X. Yang, and L. Yuan, "Helios: Real Real-Time Long Video Generation Model," *arXiv preprint arXiv:2603.04379*, 2026.

[32] H. Xi, S. Yang, Y. Zhao, C. Xu, M. Li, X. Li, Y. Lin, H. Cai, J. Zhang, D. Li *et al.*, "Sparse Video-Gen: Accelerating Video Diffusion Transformers with Spatial-Temporal Sparsity," in *Forty-second International Conference on Machine Learning*, 2025.

[33] S. Yang, H. Xi, Y. Zhao, M. Li, J. Zhang, H. Cai, Y. Lin, X. Li, C. Xu, K. Peng *et al.*, "Sparse videogen2: Accelerate video generation with sparse attention via semantic-aware permutation," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[34] B. Xu, Y. Du, Z. Liu, S. Yang, Z. Jiang, S. Yan, R. Saha, A. Pumarola, W. Wang, and P. Li, "Sparse Forcing: Native Trainable Sparse Attention for Real-time Autoregressive Diffusion Video Generation," *arXiv preprint arXiv:2604.21221*, 2026.

[35] J. Chen, Y. Zhao, J. Yu, R. Chu, J. Chen, S. Yang, X. Wang, Y. Pan, D. Zhou, H. Ling *et al.*, "Sana-video: Efficient video generation with block linear diffusion transformer," *ICLR*, 2026.

[36] J. Zhang, H. Huang, P. Zhang, J. Wei, J. Zhu, and J. Chen, "Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization," in *International Conference on Machine Learning (ICML)*, 2025.

[37] F. Liu, S. Zhang, X. Wang, Y. Wei, H. Qiu, Y. Zhao, Y. Zhang, Q. Ye, and F. Wan, "Timestep Embedding Tells: It's Time to Cache for Video Diffusion Model," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE, 2025, pp. 7353–7363.

[38] K. Gao, J. Shi, H. Zhang, C. Wang, J. Xiao, and L. Chen, "Ca2-VDM: Efficient Autoregressive Video Diffusion Model with Causal Generation and Cache Sharing," in *International Conference on Machine Learning*. PMLR, 2025, pp. 18 550–18 565.

[39] S. Ranganath, V. Menon, and A. Patnaik, "KV Cache Quantization for Self-Forcing Video Generation: A 33-Method Empirical Study," 2026.

[40] G. Xiao, Y. Tian, B. Chen, S. Han, and M. Lewis, "Efficient Streaming Language Models with Attention Sinks," in *The Twelfth International Conference on Learning Representations*, 2023.

[41] Z. Zhang, Y. Sheng, T. Zhou, T. Chen, L. Zheng, R. Cai, Z. Song, Y. Tian, C. Ré, C. Barrett, W. Zhangyang, "Atlas", and C. Beidi, "H2o: Heavy-hitter oracle for efficient generative inference of large language models," *Advances in Neural Information Processing Systems*, vol. 36, pp. 34 661–34 710, 2023.

[42] G. Xiao, J. Tang, J. Zuo, S. Yang, H. Tang, Y. Fu, S. Han *et al.*, "DuoAttention: Efficient Long-Context LLM Inference with Retrieval and Streaming Heads," in *The Thirteenth International Conference on Learning Representations*, 2024.

[43] B. Zeng, F. Ren, J. Zhang, X. Gu, K. Chen, L. Shou, and H. Li, "HybridKV: Hybrid KV Cache Compression for Efficient Multimodal Large Language Model Inference," *arXiv preprint arXiv:2604.05887*, 2026.

[44] K. Li, Z. Chen, C.-Y. Yang, and J.-N. Hwang, "Memory-Efficient Visual Autoregressive Modeling with Scale-Aware KV Cache Compression," in *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025.

[45] Z. Qin, Y. Lv, M. Lin, H. Guo, Z. Zhang, D. Zou, and W. Lin, "Head-aware kv cache compression for efficient visual autoregressive modeling," *arXiv preprint arXiv:2504.09261*, 2025.

[46] Z. Huang, Y. He, J. Yu, F. Zhang, C. Si, Y. Jiang, Y. Zhang, T. Wu, Q. Jin, N. Chanpaisit, Y. Wang, X. Chen, L. Wang, D. Lin, Y. Qiao, and Z. Liu, "VBench: Comprehensive benchmark suite for video generative models," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[47] A. Polyak, A. Zohar, A. Brown, A. Tjandra, A. Sinha, A. Lee, A. Vyas, B. Shi, C.-Y. Ma, C.-Y. Chuang *et al.*, "Movie gen: A cast of media foundation models," *arXiv preprint arXiv:2410.13720*, 2024.

[48] Z. Huang, F. Zhang, X. Xu, Y. He, J. Yu, Z. Dong, Q. Ma, N. Chanpaisit, C. Si, Y. Jiang, Y. Wang, X. Chen, Y.-C. Chen, L. Wang, D. Lin, Y. Qiao, and Z. Liu, "VBench++: Comprehensive and versatile benchmark suite for video generative models," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2025.

[49] H. Zhu, M. Zhao, G. He, H. Su, C. Li, and J. Zhu, "Causal Forcing: Autoregressive Diffusion Distillation Done Right for High-Quality Real-Time Interactive Video Generation," *arXiv preprint arXiv:2602.02214*, 2026.

[50] Z. Teed and J. Deng, "RAFT: Recurrent All-Pairs Field Transforms for Optical Flow," in *European Conference on Computer Vision*, 2020, pp. 402–419.

A Chunk Discontinuity

Definition of chunk discontinuity. In Section 3 and Section 5, we use *chunk discontinuity* to quantify transitions across chunks. To ensure fairness and validity in evaluation, we define chunk discontinuity as an intuitive metric. Specifically, for a video containing F frames and generated autoregressively in K chunks, we first compute the optical flow difference between every pair of adjacent frames using *Recurrent All-Pairs Field Transforms* (RAFT) [50], a popular deep network architecture for optical flow computation. Then, we divide the average of the Top-$(K-1)$ largest values by the overall mean. The metric is defined as:

$$\delta_t = \Delta\mathrm{RAFT}(I_t, I_{t+1}), \quad t = 1, \ldots, F-1 \tag{5}$$

$$\text{Chunk Disc.} = \frac{\mathrm{Sum}\left(\mathrm{Top}_{K-1}\left(\{\delta_t\}_{t=1}^{F-1}\right)\right)/(K-1)}{\mathrm{Sum}\left(\{\delta_t\}_{t=1}^{F-1}\right)/(F-1)} \tag{6}$$

Effectiveness of chunk discontinuity. Our metric is applicable to existing autoregressive diffusion models, which generate continuous videos without scene cut. Under this setting, for a video that is temporally smooth and continuous, the optical flow difference between adjacent frames should vary relatively uniformly, resulting in a uniformly lower metric value. Conversely, a low metric value also indicates a lower average peak difference, suggesting milder temporal variation in the video. In Figure 8, we provide an example of a video with a high metric value and poor chunk continuity. The local maxima can be seen to appear regularly at fixed-interval chunk boundaries, demonstrating that our metric effectively captures discontinuities across chunks.

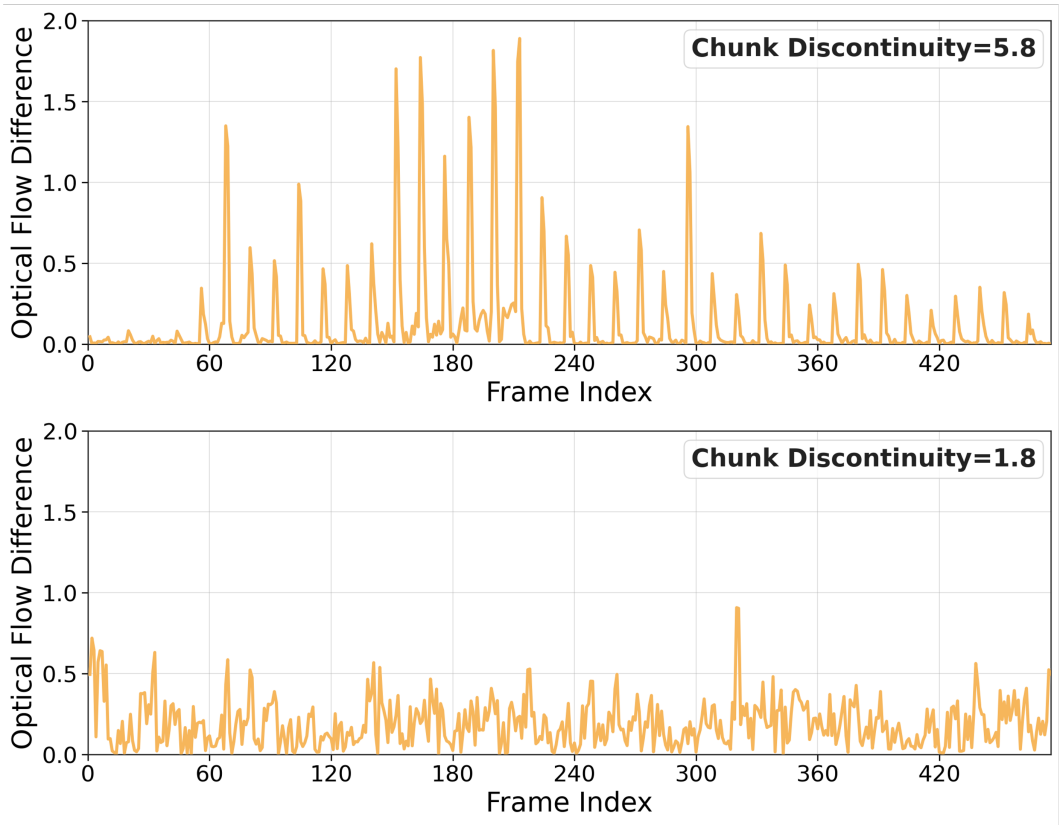

Figure 8: Case study of optical flow difference variations for a 30-second (~480 frames) video with high or low metric value.

B Attention Patterns of Various Diffusion Models

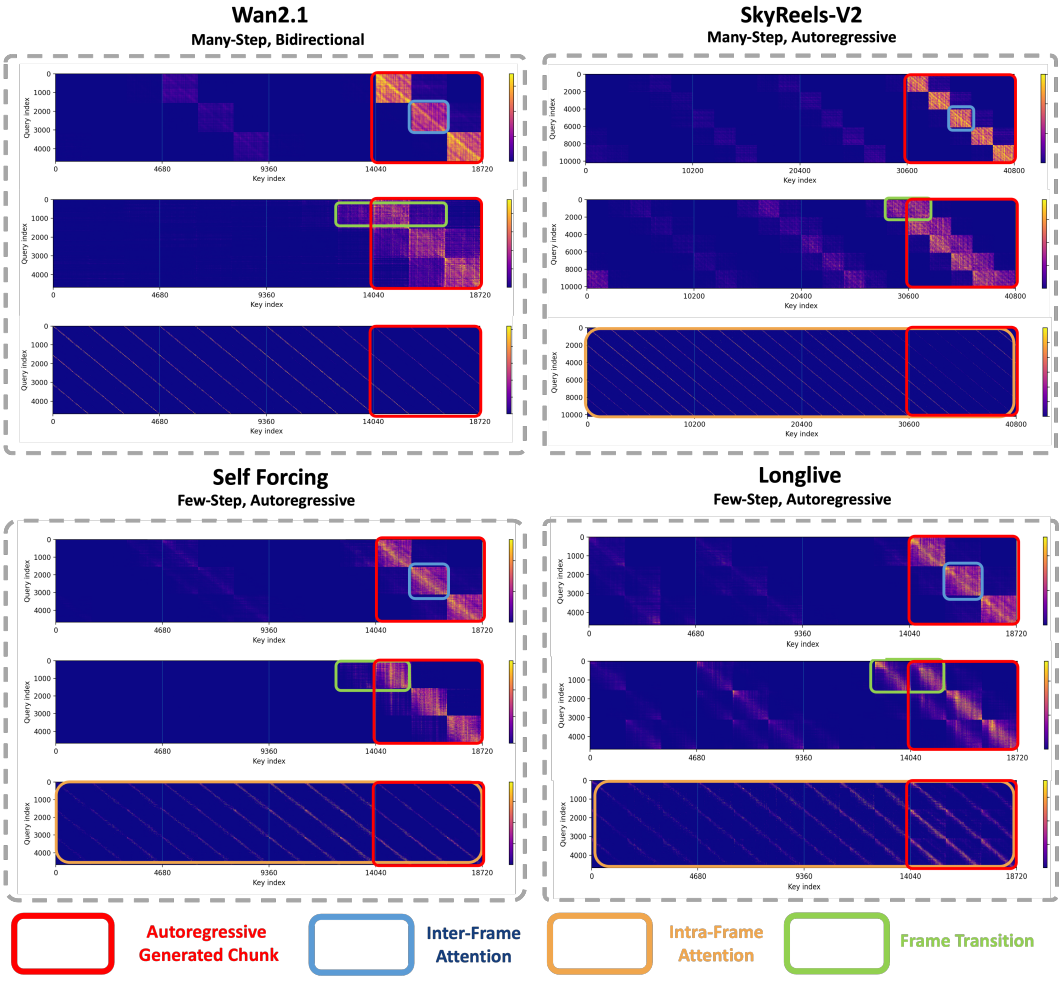

Figure 9: Attention patterns of Wan2.1 [10], SkyReels-V2 [5], Longlive [2], and Self Forcing [1].

We conduct experiments across bidirectional and autoregressive video diffusion models, including both many-step and few-step variants, such as Wan2.1 [10], SkyReels-V2 [5], LongLive [2], and Self Forcing [1], as shown in Figure 9. We find that this spatiotemporal functional specialization of attention heads is a common property across these models, where different heads are respectively responsible for inter-frame and intra-frame attention. Given that mainstream autoregressive video models are typically derived from bidirectional teacher models [1, 5, 4], we hypothesize that this property is inherited from the teacher models.

However, autoregressive video models exhibit several distinctive features. First, they learn smooth transitions from the preceding frame. Due to their autoregressive generation process (which follows a Markovian chain), the current block must acquire transition information from previous frames. Our visualization shows that such transition information is primarily concentrated in the transition anchor frame rather than the full history frames. By contrast, in bidirectional models such as Wan, these transitions are typically modeled through bidirectional attention over a local temporal range. Second, at each generation step, autoregressive video models condition on previously generated frames, which may contain substantial redundancy. Unlike bidirectional models, autoregressive models do not generate all frames in a single pass and do not require the query and key states to have identical lengths, making KV cache compression over these redundant historical frames a feasible design choice.

C User Study

Setup. To verify whether the quantitative benchmark results align with human perception, we conducted a user study with 12 participants. Each participant was presented with 15 video groups, where each group contained videos generated by different methods, including the base models (Self Forcing [1] and LongLive [2]), Dummy Forcing [25], and FORCING-KV (ours). The videos covered 5-second and 30-second generations from Self Forcing and LongLive, and were randomly sampled from the full set of 946 videos in the VBench [46, 48] benchmark. To avoid positional bias, the videos were randomly arranged as *left*, *middle*, and *right*. In total, we collected 540 evaluations (12 participants × 15 video groups × 3 videos).

Evaluation protocol. Following the previous evaluation protocol [12], we ask the participants to evaluate each video according to three key criteria:

- **Visual Quality.** This criterion captures the overall quality and appeal of each video by jointly considering factors such as visual fidelity, coherence, motion quality, and the subjective viewing experience.
- **Dynamic Degree.** This criterion measures the naturalness, richness, and engagement of motions and changes in the video. Participants assess whether the generated content exhibits realistic and diverse dynamics, rather than static or repetitive patterns.
- **Consistency.** This criterion assesses whether a video maintains visual quality and coherence throughout its entire duration, without exhibiting visual drift, artifacts, or inconsistencies.

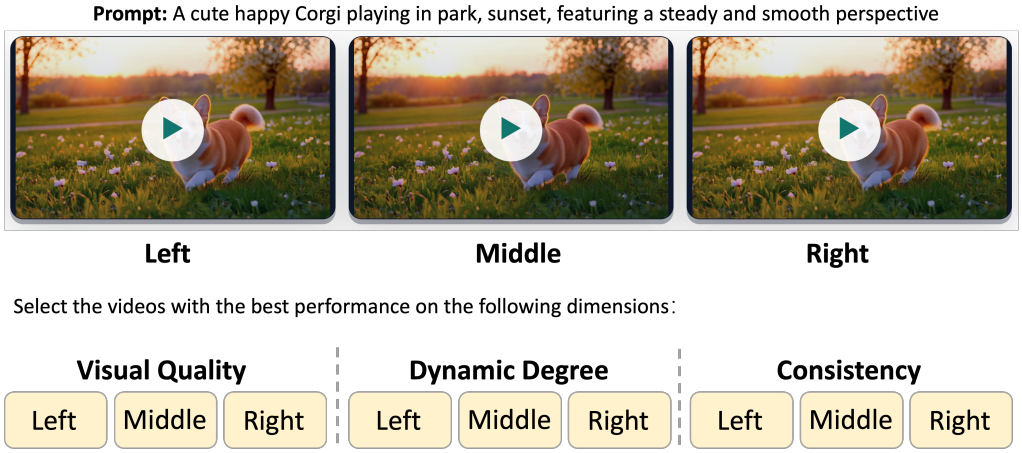

Figure 10: Protocol and screenshot of the user study.

D Results on Krea-Realtime-14B

We further extend our method to a larger model scale to validate its effectiveness. Specifically, we evaluate on Krea-Realtime-14B [27], a 14B model trained with the *Self Forcing* paradigm. We use only the offline inference version without introducing any additional optimizations. The results in Table 4 demonstrate that our method remains effective on larger-scale models.

E Interactive Video Generation

We further extend our method to interactive video generation by applying KV cache compression separately to the video segment associated with each prompt, and summarize the resulting quality and efficiency in Table 5. Specifically, we use the interactive prompts provided by LongLive [2], where each prompt group contains 6 progressively evolving descriptions, each corresponding to a 10-second video segment. We evaluate the quality of the entire generated video using VBench-Long [48]. The results show that our method achieves improvements in both quality (79.52 vs. 78.63) and inference speed (26.35 fps vs. 23.07 fps).

Method	Efficiency Metrics		Core Metrics		General Metrics		
	FPS↑	Speedup↑	Chunk Disc.↓	Dynamic Degree↑	Quality Score↑	Semantic Score↑	Total Score↑
Full KV	4.13	1.00×	1.7	77.78	85.02	81.57	84.33
FORCING-KV (Ours)	5.22	**1.26**×	1.9	73.61	85.35	81.55	**84.59**

Table 4: VBench results on 5-second video generation with Krea-Realtime-14B.

Methods	FPS↑	Quality Score						
		Total↑	Imaging Quality↑	Subject Consistency↑	Background Consistency↑	Motion Smoothness↑	Dynamic Degree↑	Aesthetic Quality↑
Longlive	23.07	78.63	69.38	98.03	96.21	99.14	26.39	59.42
Dummy Forcing	25.74	78.04	69.42	96.30	95.01	98.57	30.28	59.80
FORCING-KV	**26.35**	**79.52**	70.13	97.51	95.89	98.89	36.11	60.58

Table 5: VBench-Long results on 60-second interactive video generation with Longlive.

Methods	Efficiency Metrics		Performance Metrics		
	FPS↑	Speedup↑	Chunk Disc.↓	Dynamic Degree↑	Total Score↑
LongLive - 60s					
FORCING-KV	26.71	1.30×	2.5	43.56	80.43
FORCING-KV + FP8	28.05	1.37×	2.5	43.27	80.41
Self Forcing - 30s					
FORCING-KV	26.65	1.50×	2.7	52.23	80.07
FORCING-KV + FP8	27.55	1.55×	2.8	52.15	80.06

Table 6: Quality and efficiency results with FP8 quantization.

F Seamless Integration of Quantization

We further incorporate FP8 quantization [36] into our KV cache compression method. This optimization reduces the computational cost of attention module by leveraging FP8 attention kernels tailored to the NVIDIA Hopper architecture, which further boosts throughput with minimal performance drop, as shown in Table 6. This shows that our method is compatible with other acceleration techniques.

G More Ablation Study Results

In Section 5.3, we present the ablation results of LongLive on 30-second videos. For completeness, Tables 7 and 8 further report the ablation results on Self Forcing and LongLive for 5-second videos. The overall trends **remain consistent**. Regarding head profiling, our simple profiling strategy is already sufficient to closely approach manual profiling, while significantly outperforming the unprofiled random baseline. For KV cache compression, removing the transition anchor frame (*w/o static cache*) leads to severe chunk discontinuity, which further degrades the overall score. In contrast, removing the KV cache for dynamic heads (*w/o dynamic cache*) mainly causes a loss in dynamics, while having little effect on chunk continuity and the overall score. Since 5-second videos do not accumulate sufficient context, these effects are less pronounced than those observed on 30-second videos.

Method	Chunk Disc.↓	Dynamic Degree↑	Total Score↑
Head Profiling			
FORCING-KV	2.1	45.56	83.22
Random Profiling	2.5	41.94	82.82
Human Profiling	2.2	47.22	83.31
KV Cache Compression			
FORCING-KV	2.1	45.56	83.22
w/o static-head cache	3.9	42.22	82.60
w/o dynamic-head cache	2.3	43.33	83.08

Table 7: Ablation study on KV cache compression and head profiling strategies on LongLive - 5s.

Method	Chunk Disc.↓	Dynamic Degree↑	Total Score↑
Head Profiling			
FORCING-KV	2.1	69.17	83.98
Random Profiling	3.0	59.72	82.76
Human Profiling	2.2	71.39	84.10
KV Cache Compression			
FORCING-KV	2.1	69.17	83.98
w/o static-head cache	3.4	65.28	83.73
w/o dynamic-head cache	2.8	62.50	83.56

Table 8: Ablation study on KV cache compression and head profiling strategies on Self Forcing - 5s.

H Implementation Details

Baselines. We mainly compare our method with KV cache compression approaches for large language models, StreamingLLM [40], and with Dummy Forcing [25], a representative KV cache compression method for AR diffusion models. For StreamingLLM, we implement a naive frame-level strategy that retains 3 sink frames and 4 recent frames in the historical KV cache. For Dummy Forcing, following the original paper, we preserve the KV cache of the local region for local heads and adjust the history cache length L for neighbor heads. For an aggressive variant, we follow the official repository and set $L = 1$. For a conservative variant, we retain $L = 6$ for Self Forcing [1] and $L = 2$ for LongLive [2], corresponding to their original window sizes. A larger value of L provides a broader visible context, which can improve performance at the cost of reduced speed.

Our Method. Our method consists of two components, head profiling and hybrid KV cache compression, corresponding to the hyperparameter α and the compression ratio r for dynamic heads, respectively. For head profiling, we perform offline head classification using a single prompt with $\alpha = 0.8$, and the entire procedure can be completed within a few minutes. For hybrid KV cache compression, we retain transition frames for static heads, while setting the default compression ratio for dynamic heads to $r = 0.3$. Specifically, each historical frame in the KV cache is uniformly divided into $n = 6$ contiguous segments, segment-wise similarity is computed between adjacent frames, and the retained segments are then determined according to the compression ratio r. To avoid the lack of adjacent frames at the beginning of generation, we apply KV cache compression starting from the second autoregressive step. We preserve the sink frame similar to Dummy Forcing [25].

I VBench Scores Across All Dimensions

We further provide the detailed 16-dimension VBench [46] scores of short videos in Table 2. Specifically, on individual metrics, FORCING-KV achieves improved performance over its base model in dynamic degree, overall consistency, appearance style, and total score with reduced KV cache budget. We attribute this result primarily to the fact that FORCING-KV preserves the context required by each type of head during inference, without sacrificing either local transition information or historical contextual information. Another possible factor is that existing autoregressive models may still be imperfect in their ability to utilize distant context effectively.

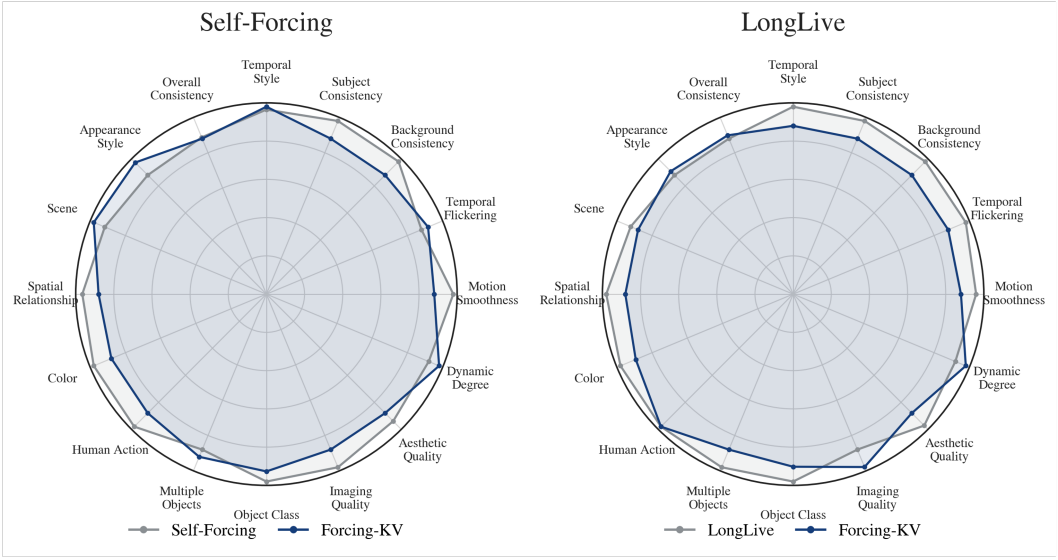

Figure 11: Visualization of VBench [46] scores. We compare FORCING-KV with its base model. FORCING-KV achieves near-lossless performance in terms of total score, and further outperforms the baseline on metrics such as dynamic degree, overall consistency, and appearance style, demonstrating its advantage.

J Where Are Static and Dynamic Heads Located?

We consider the head distribution in AR diffusion models to be an interesting research problem, and visualize the distribution of heads across layers in Figure 12. Overall, dynamic heads constitute approximately 60% of all heads, with a noticeably higher proportion in the middle layers (layer indices 13, 15, and 17). We hypothesize that this pattern arises because layers near the input and output focus more on extracting structured information to preserve local video quality, whereas the intermediate layers make heavier use of contextual information for rich feature refinement, improving detail consistency and temporal dynamics.

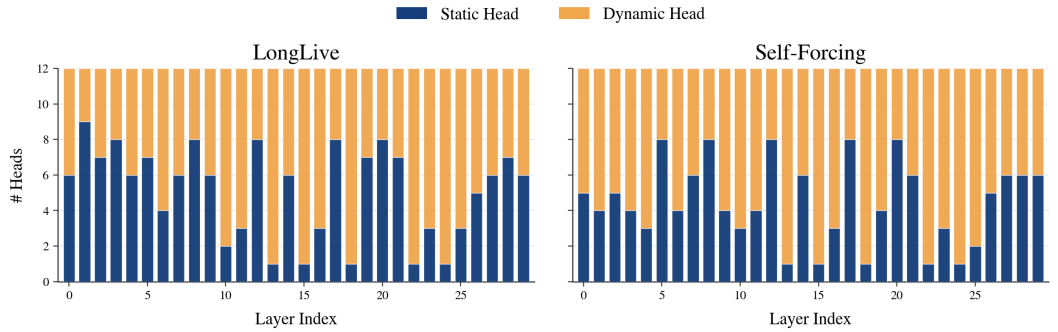

Figure 12: Head distribution across layers.

K Trend of the Proportion of Self-Attention

In Figure 6, we show how the total runtime increases as the attention window size and resolution grow. As a complementary analysis, we further break down the proportion of time consumed by self-attention. As the sequence length grows, self-attention gradually occupies a larger share of the overall Transformer block (24% to 61% and 61% to 89%), making KV cache compression increasingly beneficial.

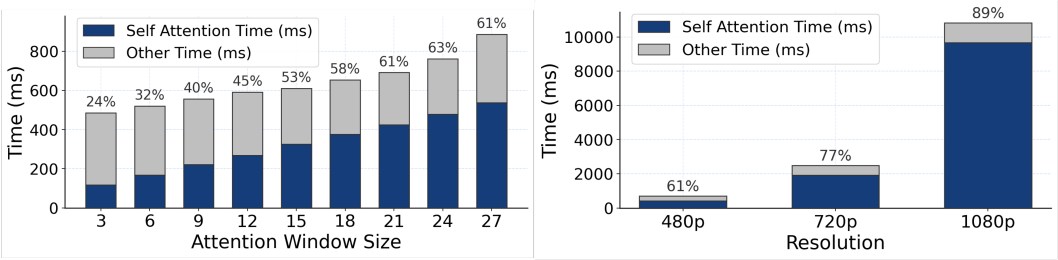

Figure 13: Variation in the proportion of total runtime occupied by self-attention.

L Quality Examples

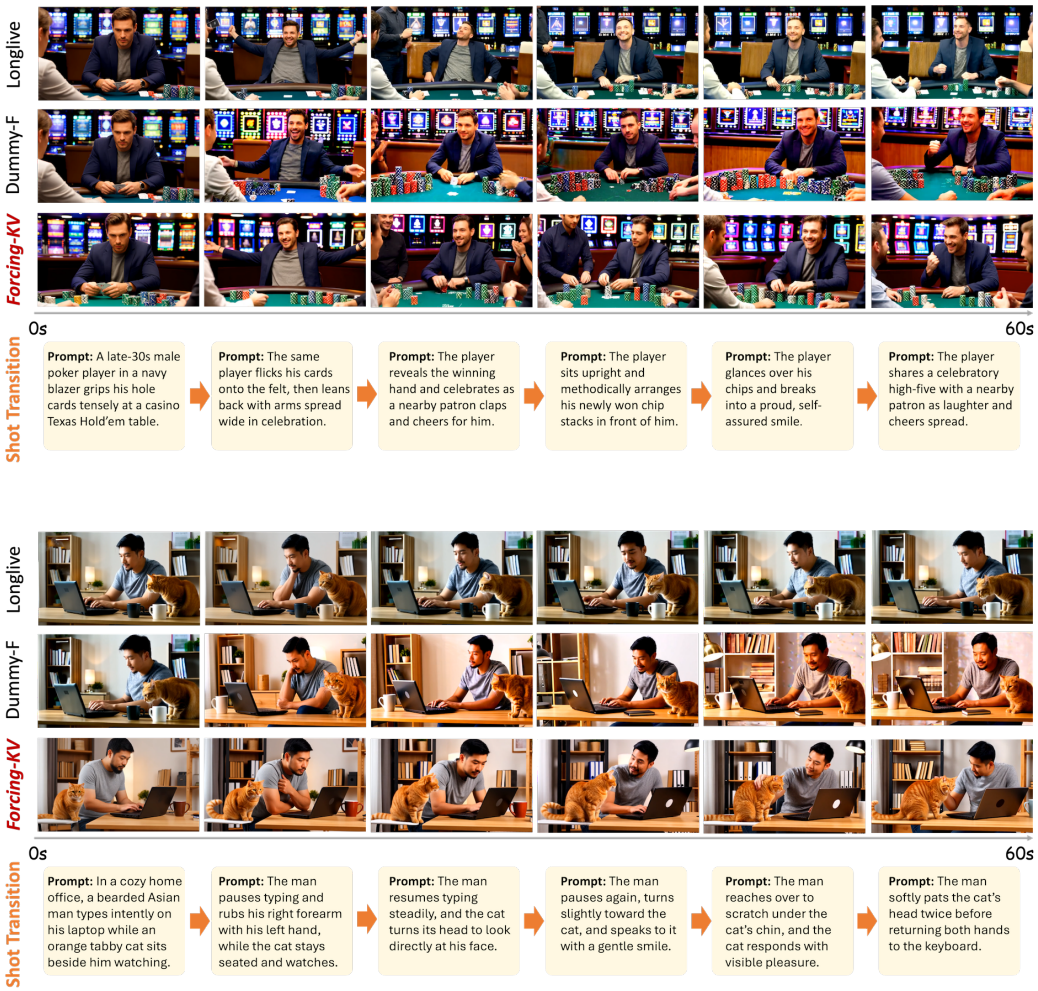

Figure 14: Quality example of 60-second interactive video on Longlive.

Figure 15: Quality examples on Longlive 5s and 30s.

Figure 16: Quality examples on Self Forcing 5s and 30s.